

Git Branching Policy – PingELZ / CIAM Platforms

Title	Git Branching Policy – PingELZ / CIAM Platforms
Version	v1.0
Status	DRAFT
Summary	This document details a clear, lowconflict Git branching strategy that supports parallel, independent releases, enables any project to promote to Production at any time, and maintains auditability, stability, and changecontrol compliance.
Technical Owner	Jelika Pahwa Praveen Chandrasekaran
Approval	

- 1. Purpose
- 2. Strategy (What we do and why)
 - 2.1 2.1 Core Principles
 - 2.2 2.2 Why we don't use classic "GitFlow develop release main"
- 3. Branch Model (Definitions + Rules)
 - 3.1 3.1 LongLived Branch
 - 3.2 3.2 Release Branches (Deployable units)
 - 3.3 3.3 Feature Branches (Shortlived)
 - 3.4 3.4 Hotfix Branches (Emergency)
- 4. Environment Promotion Strategy (Standard flow)
 - 4.1 4.1 Standard promotion path (per release branch)
 - 4.2 4.2 NonProd coordination
- 5. Implementation Playbook (How we operate day-to-day)
 - 5.1 5.1 Creating a release branch (per release)
 - 5.2 5.2 Developing within a release
 - 5.3 5.3 Promoting a release through environments
- 6. Critical Scenario Handling
 - 6.1 6.1 Policy rule (non-negotiable)
 - 6.2 6.2 Correct approach when SIT has R1, R2, R3
 - 6.3 6.3 Freezing the "Prod candidate" on R3
 - 6.4 6.4 Optional: "ProdIntent" branch for extra control (Cherrypick model)
- 7. Hotfix Implementation (Prod incident)
- 8. Controls (Branch protection & governance)
 - 8.1 8.1 main/master controls
 - 8.2 8.2 Release branch controls
- 9. Roles & Responsibilities (Implementation)
- 10. Diagrams
 - 10.1 10.1 Standard release flow
 - 10.2 10.2 "3 releases in SIT, only R3 to Prod"
 - 10.3 10.3 Optional "ProdIntent" cherry-pick model
- 11. Practical Checklists
 - 11.1 11.1 Release readiness checklist (for R3 going to Prod)
 - 11.2 11.2 Deployment hygiene checklist
- 12. Deployment & PR Governance Rules
 - 12.1 12.1 Deployment Timing & Discipline
 - 12.2 12.2 Explicit Prohibitions
 - 12.3 12.3 Summary

1. Purpose

Provide a branching strategy that supports **random, parallel releases** (any project can go to Prod anytime) while maintaining:

- **Traceability** (what code is in which environment)
- **Isolation** (multiple releases don't collide)
- **Governance** (controlled merges, repeatable evidence)
- **Operational stability** (reduce late-night merge conflicts and rollout risk)

2. Strategy (What we do and why)

2.1 Core Principles

- **Production baseline is the source of truth:** main/master represents **what is running in PROD**.
- **Release branches are the unit of deployment:** deployments happen from release/* branches (not from a shared "SIT branch").

- **Environment branch**: environments are **validation lanes**, not code baselines.
- **Do not bundle unrelated work**: keep releases small and coherent.
- **Separate infra changes from app/config changes** to reduce rollback blast radius.
- **Hotfixes start from Prod baseline** and must be propagated back into active releases.

2.2 Why we don't use classic "GitFlow develop release main"

Because with random releases, a shared develop becomes a conflict hotspot and forces unrelated work to integrate early.

3.Branch Model (Definitions + Rules)

3.1 LongLived Branch

main / master

- Must match PROD.
- Updated **only after successful Prod deployment**.
- Protected: PR-only merges; no force-push; no rewrite history; no delete.

3.2 Release Branches (Deployable units)

release/<release-name> or release/<project>/<version>

- One branch per release stream.
- Can exist in parallel (R1, R2, R3).
- Deployments to environments occur from these release branches.

3.3 Feature Branches (Shortlived)

feature/<ticket>-<short-desc>

- Created from the target release/* branch (recommended).
- Merged back into the same release branch via PR.

3.4 Hotfix Branches (Emergency)

hotfix/<incident>/<version>

- Created from main/master for urgent Prod fixes.
- Deployed quickly, then merged back to main/master, and applied to active release branches.

4. Environment Promotion Strategy (Standard flow)

4.1 Standard promotion path (per release branch)

- release/* is promoted through:
 - **dev eSIT SIT SVP UAT PROD**
- After PROD is successful:
 - merge release/* main/master (PR controlled).

This aligns with internal patterns where master/main is in sync with Prod and release branches are used for deployments.

4.2 NonProd coordination

- Align deployments to the shared **NonProd Release Calendar** process.

5. Implementation Playbook (How we operate day-to-day)

5.1 Creating a release branch (per release)

1. Create release/R1 (or release/<project>/<version>) from main/master.

2. Only changes intended for that release merge into that release branch. (Recommended)

5.2 Developing within a release

1. Create feature branches from the target release branch:
 - feature/JIRA-1234-xyz PR release/Rx
2. PR checks & reviews required (see Controls).

5.3 Promoting a release through environments

1. Deploy from release/Rx into eSIT.
2. Promote the **same release branch commit** into SIT, SVP, UAT, then PROD.
3. Merge into main/master after successful PROD deployment.

Operational guidance to reduce risk: keep a stability window between UAT and Prod where feasible.

6. Critical Scenario Handling

SIT has 3 Release Branches (R1, R2, R3) and only R3 is ready for Prod

6.1 Policy rule (non-negotiable)

1. Promotion to UAT/Prod must reference a specific release branch commit (or a tag on it).
2. Never promote “whatever is currently in SIT.”

6.2 Correct approach when SIT has R1, R2, R3

- Promote only **R3** by deploying from **release/R3** to SVP UAT PROD (not from “SIT state”). (Recommended)
- R1 and R2 can remain in SIT for testing; they are not automatically included in UAT/Prod if promotions are branch-based. (Recommended)

6.3 Freezing the “Prod candidate” on R3

To avoid last-minute drift:

- Freeze merges to release/R3 after approval window (recommended)
- Use tags for traceability (recommended)

6.4 Optional: “ProdIntent” branch for extra control (Cherrypick model)

If SIT is noisy or frequently contains extra changes, create a controlled promotion branch (example pattern already discussed internally):

*Developers test changes in the SIT lane, then **cherry-pick only the approved changes** into a “prod-intent” release branch (e.g., release/dev00) for UAT/Prod progression.*

7. Hotfix Implementation (Prod incident)

1. Create hotfix/<incident>/<version> from main/master.
2. Deploy hotfix through required validation (UAT minimum per change controls). (Recommended)
3. Merge hotfix back to main/master.
4. Propagate hotfix into active release branches (merge/cherry-pick) to prevent regression.

Operational consideration: ensure L3/Change coverage during deployment windows to handle build/conflict issues.

8. Controls (Branch protection & governance)

8.1 main/master controls

1. PR-only merges; no direct pushes.
2. No force push / history rewrite / delete.
3. Merge requires approval by nominated maintainers (Support/Tech Lead).

8.2 Release branch controls

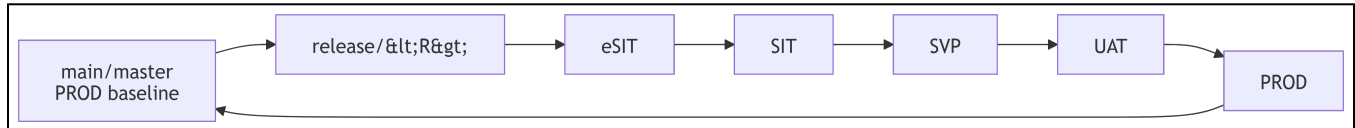
1. PR-only merges (recommended)
2. Limit who can approve "promotion-ready" state (recommended)
3. Avoid bundling multiple unrelated tenant changes in one release.
4. Keep infra and app changes separate.

9. Roles & Responsibilities (Implementation)

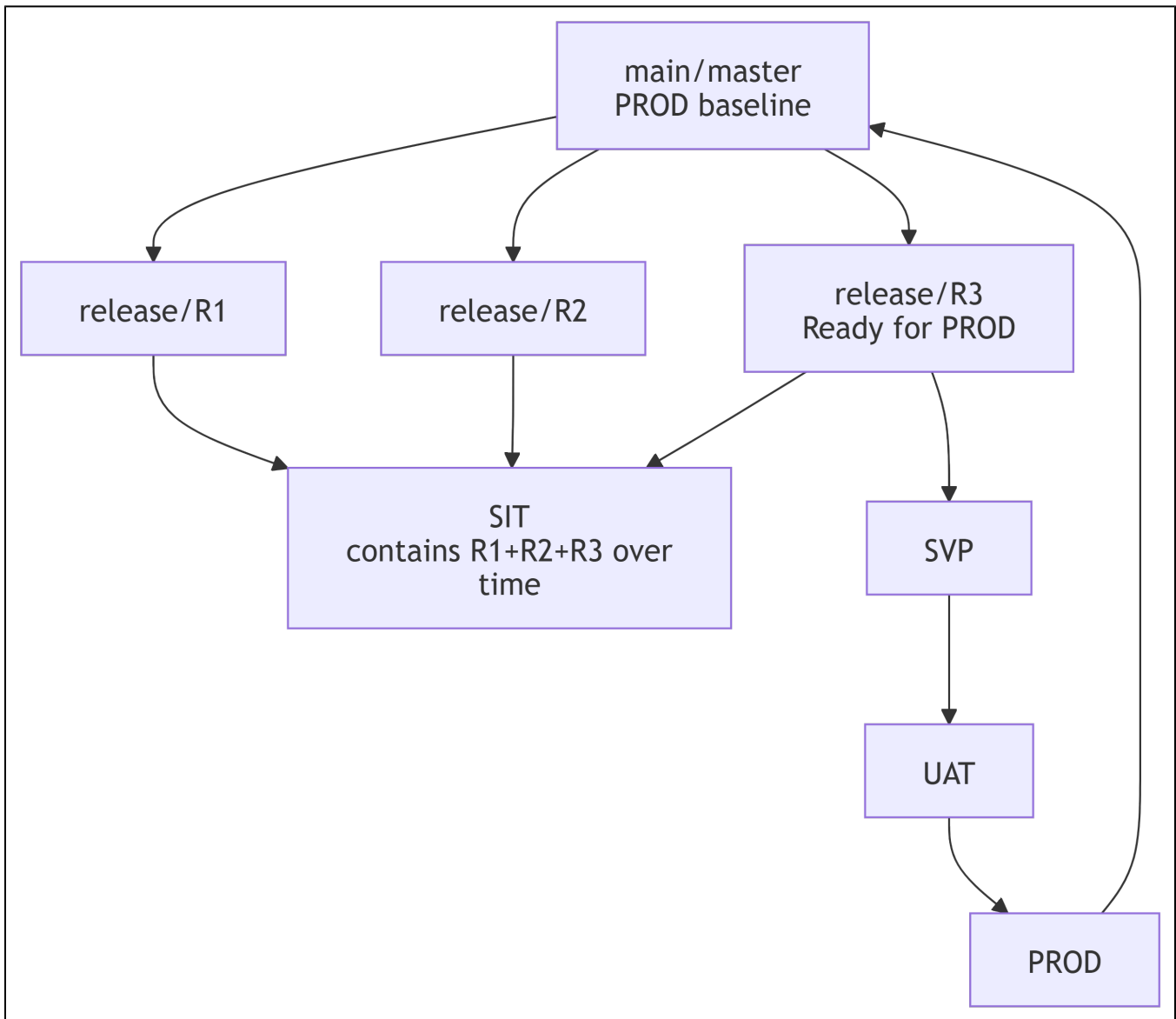
1. **Change Engineers**
 - a. Work on feature/*, raise PRs into target release/*
2. **L3 Release**
 - a. Creates release/*, controls merges to main/master, approves promotion readiness
3. **L2 / Ops**
 - a. Executes deployments and standard operational runbooks
4. **L3 / Change team**
 - a. Available during deployments for PR/build failures or conflicts (recommended based on prior issues).

10. Diagrams

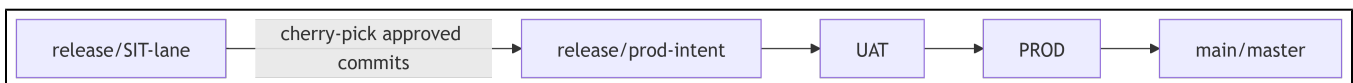
10.1 Standard release flow



10.2 "3 releases in SIT, only R3 to Prod"



10.3 Optional “ProdIntent” cherry-pick model



11. Practical Checklists

11.1 Release readiness checklist (for R3 going to Prod)

- Release branch identified: release/R3
- Promotion candidate commit identified (tag recommended)
- Freeze additional merges into release/R3
- Promote R3 commit to SVP then UAT
- UAT stability and sign-off confirmed (stability gap recommended)
- Prod deployment completed
- Merge release/R3 main/master after success

11.2 Deployment hygiene checklist

- Infra changes separated from app/config changes
- Avoid bundling unrelated changes in one deployment
- L3/Change availability during deployment window
- No last-minute PRs into the release once UAT sign-off starts (recommended)

12. Deployment & PR Governance Rules

12.1 Deployment Timing & Discipline

- Deployments must use **preapproved release branch commits or tags**
- No lastminute code changes during deployment windows
- Recommended stability window between:
 - **UAT Prod** (timebox defined by release manager)
- Infrastructure and application/config deployments must be **separate release branches**

12.2 Explicit Prohibitions

No deploying from feature branches beyond Dev

No promoting “current environment state”

No multiple PR approvals per environment

No bundling unrelated changes into one release

No forcepush or history rewrite on main/master

12.3 Summary

Environment	PR Required	Max Approvals	Deployment Source
Dev	No	N/A	feature/*
eSIT	Yes	1	release/*
SIT	Yes	1	release/*
SVP	Yes	1	same release commit
UAT	Yes	1	same release commit
Prod	Yes	1	tagged release/hotfix